Atividade 7

JOAO RICARDO ZULATO REBERTI

Setembro 2026

1 Introdução

Esta atividade é sobre navegar nos repositórios dos projetos dos compiladores atuais:GCC (GNU Compiler Collection), Clang / LLVM e Anthropic CCC (Claude's C Compiler) para verificar como foi feita a implementação da Análise Léxica nesses compiladores.

2 Implementação da Análise Léxica nos Compiladores

GCC (GNU Compiler Collection)

No GCC, a implementação do lexer (analisador léxico) não está em um único arquivo isolado no diretório principal, mas sim dividida entre a biblioteca do pré-processador e uma camada de interface.

O arquivo principal do Lexer do GCC: libcpp/lex.cc

No GCC moderno (escrito em C++), quase toda a lógica de transformar caracteres em tokens fica dentro da libcpp, que é a biblioteca do pré-processador.

Procure pela função _cpp_lex_direct ela é a função principal do código. É nela que o trabalho acontece: o código percorre o buffer, identifica se é um número, uma string ou um identificador e lida com as complexidades de escapes e trigraphs

A Interface com o Parser é feita no arquivo gcc/c-family/c-lex.cc. Enquanto a libcpp gera os tokens "brutos", o compilador precisa de uma camada que traduza esses tokens para o que o Parser de C ou C++ entende.

Ele chama a libcpp para obter o próximo token e o converte em uma estrutura que o GCC chama de tree (para literais e identificadores) ou códigos de token específicos para a gramática.

Outros arquivos relevantes para o seu projeto:

1. libcpp/include/cpplib.h: Contém as definições das estruturas de tokens (cpp_token) e os tipos de tokens que a libcpp reconhece.

2. gcc/c/c-parser.cc e gcc/cp/parser.cc: Embora sejam os analisadores sintáticos (Parsers), eles contêm funções que "consomem" os tokens vindos do lexer.

No C++, por exemplo, o lexer é muito mais complexo devido ao lookahead (olhar à frente) necessário para distinguir entre uma declaração de variável e uma chamada de função.

Clang / LLVM

O Arquivo Principal: clang/lib/Lex/Lexer.cpp

A função Lexer::LexTokenInternal é o loop principal (o grande switch) que decide se o caractere atual é o início de um comentário, um número, uma string ou um identificador.

A Definição da Interface: clang/include/clang/Lex/Lexer.h

Para entender como o resto do compilador se integra com o lexer, este é o arquivo de cabeçalho. Ele define a classe Lexer e os estados necessários para manter o rastreamento de onde o compilador está no arquivo-fonte.

No clang, o Lexer raramente é chamado isoladamente. Ele geralmente é um componente do Preprocessor. O preprocessador: clang/lib/Lex/Preprocessor.cpp

O Preprocessor é quem realmente fornece os tokens para o Parser.

Ele gerencia as macros (#define), inclusões (#include) e condicionais (#if), chamando o Lexer.cpp para obter os tokens básicos antes de processar as diretivas.

Outros arquivos importantes no diretório lib/Lex/:

1. LiteralSupport.cpp: Contém a lógica para interpretar literais (ex: converter a string "0x123" no valor numérico hexadecimal real ou lidar com sufixos como 1.0f).

2. TokenLexer.cpp: Responsável por extrair tokens que vêm de expansões de macros, e não diretamente do arquivo de texto.

3. UnicodeCharSets.h: Suporte a identificadores em UTF-8 e caracteres especiais.

Anthropic CCC (Claude's C Compiler)

Os arquivos principais da fase léxica são: src/frontend/lexer/mod.rs, src/frontend/lexer/scan.rs e os tokens estão definidos em src/frontend/lexer/token.rs.